

TRƯỜNG ĐẠI HỌC SƯ PHẠM HÀ NỘI
TRƯỜNG TOÁN HỌC VÀ CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ THÔNG TIN



CHUYÊN ĐỀ TỐT NGHIỆP DỰ ÁN CÔNG NGHỆ

ĐỀ TÀI:

**XÂY DỰNG ỨNG DỤNG QUẢN LÝ VÀ TĂNG TỐC TẢI
XUỐNG
FLOW DOWNLOAD MANAGER ĐA NỀN TẢNG**

**Giảng viên hướng
dẫn:**

ThS. Trần Hải Long

Sinh viên thực hiện:

Lưu Lâm Công - 715105032

XÁC NHẬN CỦA GIẢNG VIÊN HƯỚNG DẪN

Hà Nội, ngày.....tháng.....năm 2026

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

ThS. Trần Hải Long

LỜI CAM ĐOAN

Em xin cam đoan nội dung và những kết quả của chuyên đề tốt nghiệp này là do bản thân em tự nghiên cứu dưới sự hướng dẫn của ThS. Trần Hải Long. Trong toàn bộ nội dung của chuyên đề, những nội dung được trình bày là của riêng em hoặc được tổng hợp từ nhiều nguồn tài liệu khác một cách trung thực. Tất cả các tài liệu tham khảo được sử dụng đều được trích dẫn nguồn gốc rõ ràng ở phần cuối của báo cáo.

Em xin cam đoan những lời trên là sự thật. Nếu sai em xin hoàn toàn chịu trách nhiệm.

Hà Nội, ngày.....tháng.....năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Lưu Lâm Công

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Trần Hải Long, người đã tận tình hướng dẫn, định hướng chuyên môn và hỗ trợ em trong suốt quá trình nghiên cứu và hoàn thành chuyên đề tốt nghiệp này.

Đồng thời, em cũng xin gửi lời cảm ơn chân thành tới quý thầy cô khoa Công nghệ thông tin - Trường Đại học Sư phạm Hà Nội đã truyền đạt kiến thức và tạo điều kiện học tập tốt nhất cho em trong những năm học qua.

Mặc dù đã nỗ lực hoàn thành chuyên đề một cách nghiêm túc, song kết quả nghiên cứu khó tránh khỏi những hạn chế thiếu sót. Em rất mong nhận được sự đóng góp ý kiến từ quý thầy cô để đề tài được hoàn thiện hơn.

Hà Nội, ngày.....tháng.....năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Lưu Lâm Công

MỤC LỤC

(Vui lòng nhấp chuột phải vào đây trong Word và chọn "Update Field" để tự động cập nhật Mục lục theo số trang mới)

DANH MỤC CHỮ VIẾT TẮT

Chữ viết tắt	Diễn giải
CSDL	Cơ sở dữ liệu
KMP	Kotlin Multiplatform
UI	User Interface (Giao diện người dùng)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
HLS	HTTP Live Streaming (Luồng phát trực tuyến)
HTTP/HTTPS	HyperText Transfer Protocol / Secure
IDE	Integrated Development Environment (Môi trường phát triển tích hợp)
DI	Dependency Injection (Tiêm phụ thuộc)
JSON	JavaScript Object Notation
OS	Operating System (Hệ điều hành)
CI/CD	Continuous Integration / Continuous Deployment

DANH MỤC HÌNH VẼ

- **Hình 1.1:** Tốc độ tải xuống đa luồng so với đơn luồng truyền thống.
- **Hình 2.1:** Kiến trúc hoạt động của Kotlin Multiplatform chia sẻ mã nguồn.
- **Hình 2.2:** So sánh Jetpack Compose với giao diện XML truyền thống.
- **Hình 3.1:** Kiến trúc 3 tầng của hệ thống Flow Download Manager.
- **Hình 3.2:** Biểu đồ Use Case tổng quát cho ứng dụng Flow.
- **Hình 3.3:** Biểu đồ Use Case chi tiết của chức năng Tải xuống tập tin.
- **Hình 3.4:** Quy trình chặn link tải xuống từ Browser Extension thông qua Local Server.
- **Hình 3.5:** Mô hình các lớp dữ liệu cấu hình lưu trữ qua DataStore.
- **Hình 4.1:** Giao diện Trang chủ (Home) của Flow Download Manager trên nền tảng Desktop.
- **Hình 4.2:** Giao diện quản lý hàng đợi và giới hạn băng thông.
- **Hình 4.3:** Giao diện ứng dụng Flow trên hệ điều hành Android.
- **Hình 4.4:** Danh sách các chủ đề (Themes) của giao diện Cyber-Industrial.

DANH MỤC BẢNG BIỂU

- **Bảng 1.1:** So sánh các tính năng của Flow Download Manager với các ứng dụng cùng loại.
- **Bảng 3.1:** Đặc tả Use Case "Tải xuống tệp tin mới".
- **Bảng 3.2:** Các thông số kỹ thuật cấu hình trong AppSettings.
- **Bảng 4.1:** Kết quả thực nghiệm đo lường tốc độ và thời gian tải giữa Flow và Chrome Downloader.

LỜI NÓI ĐẦU

Trong kỷ nguyên số hiện nay, Internet đã trở thành một phần thiết yếu không thể thiếu trong công việc, học tập và giải trí của mỗi cá nhân. Khối lượng dữ liệu trao đổi trên mạng ngày một tăng lên nhanh chóng, kéo theo kích thước các tệp tin tải xuống (phần mềm, video chất lượng cao, tài liệu học tập, các gói cài đặt hệ thống) ngày càng lớn. Việc tải xuống các tệp tin dung lượng lớn bằng các trình duyệt web thông thường (như Google Chrome, Microsoft Edge, Mozilla Firefox) thường gặp phải các hạn chế lớn như: tốc độ tải chậm do chỉ hỗ trợ kết nối đơn luồng (single-connection), thiếu khả năng quản lý hàng đợi tải xuống hiệu quả, dễ bị lỗi và không thể tiếp tục tải lại (resume) khi mất kết nối đột ngột giữa chừng.

Từ thực tiễn đó, các ứng dụng hỗ trợ và tăng tốc tải xuống đóng vai trò vô cùng quan trọng. Các phần mềm này sử dụng kỹ thuật phân tách tệp tin thành nhiều phần (chunks) và tải xuống đồng thời trên nhiều luồng khác nhau, giúp tận dụng tối đa băng thông đường truyền internet để tăng tốc độ tải lên gấp nhiều lần. Tuy nhiên, các công cụ phổ biến hiện nay như Internet Download Manager (IDM) lại là phần mềm độc quyền trả phí, chỉ hoạt động trên hệ điều hành Windows và chứa các đoạn mã theo dõi (telemetry) người dùng. Trong khi đó, các giải pháp miễn phí khác thường bị giới hạn tính năng hoặc chứa nhiều quảng cáo phiền toái.

Nhận thấy nhu cầu cấp thiết về một công cụ tải xuống mạnh mẽ, đa nền tảng, hoàn toàn miễn phí, mã nguồn mở và đảm bảo tính riêng tư tuyệt đối cho người dùng, tác giả đã lựa chọn đề tài: "**Xây dựng ứng dụng quản lý và tăng tốc tải xuống Flow Download Manager đa nền tảng**". Ứng dụng được thiết kế dựa trên các công nghệ lập trình tiên tiến nhất hiện nay như Kotlin Multiplatform và Compose Multiplatform, cho phép ứng dụng chạy mượt mà trên cả máy tính để bàn (Windows, macOS, Linux) và thiết bị di động (Android) với chỉ một bộ mã nguồn dùng chung duy nhất, mang lại trải nghiệm đồng nhất và tối ưu hóa tài nguyên phần cứng tốt nhất.

CHƯƠNG I: GIỚI THIỆU

1.1. Bài toán tải xuống và quản lý tập tin trên môi trường mạng

Khi người dùng tải xuống một tập tin thông qua trình duyệt web mặc định, trình duyệt sẽ gửi một yêu cầu GET đơn lẻ đến máy chủ chứa tập tin và thiết lập một kết nối duy nhất để kéo toàn bộ luồng dữ liệu về. Cách tiếp cận này bộc lộ những nhược điểm rõ rệt:

- **Giới hạn băng thông đơn luồng:** Các máy chủ thường giới hạn băng thông tối đa trên mỗi kết nối để đảm bảo tính công bằng. Kết nối đơn luồng sẽ không thể khai thác tối đa băng thông thực tế của đường truyền mạng người dùng.
- **Dễ bị ngắt quãng:** Khi kết nối internet gặp sự cố tạm thời, luồng tải của trình duyệt thường bị lỗi hoàn toàn, buộc người dùng phải tải lại từ đầu, gây lãng phí dung lượng đường truyền và thời gian.
- **Thiếu khả năng phân loại và lập lịch:** Người dùng khó có thể tổ chức quản lý tập tin theo các danh mục cụ thể (tài liệu, video, nhạc, nén) hoặc hẹn giờ tải xuống vào ban đêm khi băng thông rỗi.
- **Hạn chế với luồng phát trực tuyến (m3u8/HLS):** Các trình duyệt không hỗ trợ tải trực tiếp các luồng video phát trực tuyến phân đoạn (HLS) vốn đang rất phổ biến hiện nay.

Để giải quyết triệt để các vấn đề trên, ứng dụng **Flow Download Manager** được xây dựng với mục tiêu cung cấp giải pháp tải xuống đa luồng mạnh mẽ, tự động phân đoạn tập tin một cách thông minh, hỗ trợ phục hồi tiến trình bị gián đoạn và tích hợp sâu rộng vào trình duyệt thông qua tiện ích mở rộng tự phát triển.

1.2. Mục đích và phạm vi nghiên cứu

Mục đích nghiên cứu:

- Nghiên cứu nguyên lý hoạt động của giao thức HTTP/HTTPS, đặc biệt là cơ chế phân tách phạm vi tải xuống Range Requests (HTTP Header Range và Content-Range) để tải đồng thời nhiều phần của một tập tin.
- Nghiên cứu kiến trúc phát triển đa nền tảng hiện đại Kotlin Multiplatform (KMP) để tối đa hóa khả năng tái sử dụng mã nguồn giữa các hệ điều hành khác nhau.
- Nghiên cứu cơ chế giao tiếp liên tiến trình (IPC) giữa tiện ích mở rộng của trình duyệt (Browser Extension) và ứng dụng gốc (Native App) thông qua một máy chủ cục bộ (Local HTTP Server) chạy ngầm để bắt link tự động.

- Nghiên cứu cơ chế tải xuống, ghép nối và xử lý luồng phát trực tuyến HLS (HTTP Live Streaming) dựa trên định dạng m3u8.

Phạm vi nghiên cứu:

- Phát triển và hoàn thiện công cụ tải xuống Flow Download Manager chạy tốt trên các hệ điều hành Desktop (Windows, macOS, Linux) và di động (Android).
- Phát triển Tiện ích mở rộng tương thích với các trình duyệt nhân Chromium (Google Chrome, Microsoft Edge, Brave) sử dụng Manifest V3.
- Xây dựng hệ thống quản lý hàng đợi, giới hạn băng thông chủ động cho từng tác vụ tải xuống.

1.3. Khảo sát hiện trạng ứng dụng

Để làm rõ thế mạnh của Flow Download Manager, tác giả đã tiến hành khảo sát và so sánh trực tiếp các công cụ tải xuống phổ biến nhất trên thị trường hiện nay bao gồm: Internet Download Manager (IDM), Neat Download Manager và Trình tải xuống mặc định của trình duyệt Google Chrome.

Bảng 1.1: So sánh các tính năng của Flow Download Manager với các ứng dụng cùng loại

Tiêu chí	Chrome Downloader	Internet Download Manager (IDM)	Neat Download Manager	Flow Download Manager
Tải đa luồng	Không hỗ trợ (Đơn luồng)	Có hỗ trợ (Tối đa 32 luồng)	Có hỗ trợ (Tối đa 10 luồng)	Có hỗ trợ (Tối đa 16 luồng)
Giá thành & Bản quyền	Miễn phí	Trả phí (Dùng thử 30 ngày)	Miễn phí	Miễn phí (Mã nguồn mở hoàn toàn)
Nền tảng hỗ trợ	Windows, macOS, Linux, Android	Chỉ hỗ trợ Windows	Windows, macOS	Windows, macOS, Linux, Android
Quản lý hàng đợi	Không hỗ trợ	Rất tốt	Hạn chế	Rất tốt (Đặt lịch, giới hạn tốc độ)
Độ an toàn riêng tư	Trung bình (Có telemetry)	Không rõ (Đóng gói mã nguồn)	Không rõ (Đóng gói mã nguồn)	Cao nhất (Zero telemetry, 100% offline)
Tải video HLS/m3u8	Không hỗ trợ	Có hỗ trợ	Hạn chế	Có hỗ trợ trực tiếp

Giao diện thiết kế	Đơn giản, mặc định	Cổ điển, ít cập nhật	Đơn giản	Cyber-Industrial (Đột phá, 6 themes)
---------------------------	--------------------	----------------------	----------	---

Qua bảng so sánh trên, có thể thấy Flow Download Manager nổi lên như một giải pháp tối ưu nhờ sự kết hợp giữa hiệu năng tải xuống vượt trội của IDM, tính năng mã nguồn mở hoàn toàn miễn phí, hỗ trợ đa nền tảng thực sự và phong cách thiết kế UI đột phá.

1.4. Quy trình quản lý và phát triển dự án

Dự án được quản lý và phát triển theo mô hình Agile/Scrum rút gọn để đảm bảo tính linh hoạt:

- **Quản lý mã nguồn:** Sử dụng Git và lưu trữ trên nền tảng GitHub. Mã nguồn được chia thành các nhánh chức năng rõ ràng (main cho sản xuất, dev cho tích hợp thử nghiệm, và các nhánh feature/*).
- **Hệ thống CI/CD:** Cấu hình GitHub Actions với workflow build-check.yml tự động kích hoạt kiểm thử đơn vị (Unit Tests) trên các module cốt lõi như :downloader:core và :shared:utils, chạy trình kiểm tra lỗi Android Lint bất cứ khi nào có thay đổi được đẩy lên repository.
- **Phiên bản hóa:** Tự động hóa việc đánh số phiên bản dựa trên Git tag thông qua một custom plugin (git-version-plugin) tích hợp trực tiếp vào Gradle build logic.

CHƯƠNG II: NGHIÊN CỨU GIẢI PHÁP CÔNG NGHỆ

2.1. Kotlin Multiplatform (KMP)

Kotlin Multiplatform (KMP) là một công nghệ được phát triển bởi JetBrains, cho phép chia sẻ mã nguồn logic nghiệp vụ giữa các nền tảng khác nhau mà không cần biên dịch sang một định dạng trung gian. KMP biên dịch mã nguồn trực tiếp thành mã máy bản địa (native code) của từng nền tảng mục tiêu:

- Biên dịch thành **Java Bytecode** cho JVM trên Desktop và Android.
- Biên dịch thành **Objective-C/Swift framework** cho iOS/macOS thông qua trình biên dịch Kotlin/Native.
- Biên dịch thành **WebAssembly (Wasm) hoặc JavaScript** cho nền tảng Web.

Nhờ có KMP, tác giả có thể xây dựng toàn bộ thư viện tải xuống cốt lõi (downloader:core) một lần duy nhất bằng ngôn ngữ Kotlin thuần túy và sử dụng lại 100% trên cả Desktop và Android, giảm thiểu sai sót đồng bộ và tăng tốc độ phát triển dự án.

2.2. Compose Multiplatform

Compose Multiplatform là một framework xây dựng giao diện người dùng khai báo (declarative UI) dựa trên Jetpack Compose của Google. Nó cho phép các nhà phát triển sử dụng cùng một bộ API vẽ giao diện để tạo ra UI chạy trên Android, iOS, Desktop và Web.

Khác với các giải pháp lai (hybrid) khác sử dụng WebView, Compose Multiplatform vẽ trực tiếp các phần tử giao diện lên màn hình bằng thư viện đồ họa **Skia** (hoặc **Skiko** trên JVM/Native), mang lại hiệu năng vẽ cực kỳ cao, đạt tốc độ 60-120 khung hình trên giây (FPS), giúp các hiệu ứng chuyển động và thao tác cuộn trong ứng dụng Flow mượt mà như ứng dụng bản địa thuần túy.

2.3. Thư viện Ktor & OkHttp Client

Để thiết lập các kết nối mạng hiệu năng cao, ứng dụng sử dụng sự kết hợp giữa:

- **Ktor Client:** Thư viện mạng hướng bất đồng bộ (asynchronous) viết riêng cho Kotlin, hỗ trợ lập trình bằng Coroutines. Ktor được sử dụng để gọi các API kiểm tra cập nhật, tải cấu hình.
- **OkHttp Client:** Được cấu hình chuyên biệt ở tầng dưới (JVM/Android engine) cho phần download lõi. Tác giả đã cấu hình tắt tính năng dồn kênh (HTTP/2 multiplexing) và bắt buộc sử dụng HTTP/1.1 khi tải xuống tệp tin dung

lượng lớn, nhằm tránh hiện tượng tắc nghẽn đường truyền (head-of-line blocking) và tối ưu hóa tối đa tốc độ tải qua nhiều luồng song song.

2.4. Cơ chế lưu trữ Jetpack DataStore

Thay vì sử dụng SQLite đòi hỏi cấu hình phức tạp và tốn tài nguyên cho các mục đích lưu cấu hình đơn giản, Flow sử dụng thư viện **Jetpack DataStore (Preferences/Proto)**. DataStore lưu trữ dữ liệu bất đồng bộ dưới dạng các tệp JSON cục bộ, sử dụng thư viện tuần tự hóa siêu tốc `kotlinx.serialization` để chuyển đổi qua lại giữa cấu hình in-memory và file vật lý một cách an toàn, tránh hoàn toàn rủi ro gây block luồng giao diện chính (Main Thread).

2.5. Kiến trúc điều hướng Decompose

Decompose là một thư viện Kotlin Multiplatform giúp chia nhỏ mã nguồn UI thành các thành phần đóng gói độc lập gọi là **Component**. Mỗi Component tự quản lý vòng đời (lifecycle), trạng thái (state), và cơ chế tiêm phụ thuộc (DI) riêng thông qua `ComponentContext`.

Đặc biệt, Decompose cung cấp hệ thống điều hướng (navigation) cực kỳ mạnh mẽ dưới dạng State Keeper, giúp ứng dụng Flow lưu lại trạng thái màn hình hiện tại chính xác ngay cả khi bị xoay màn hình điện thoại hoặc khi thu nhỏ ứng dụng xuống khay hệ thống (System Tray).

2.6. Tiện ích mở rộng trình duyệt (Browser Extension Manifest v3)

Để tự động phát hiện và đánh chặn các liên kết tải xuống từ trình duyệt của người dùng, tác giả đã xây dựng một tiện ích mở rộng tương thích với Manifest V3. Tiện ích này chạy ngầm và thực hiện:

1. Đăng ký bộ lắng nghe sự kiện tải xuống `chrome.downloads.onCreated`.
2. Khi phát hiện tệp tin có kích thước lớn hơn 1MB, tiện ích sẽ tạm dừng trình tải mặc định của trình duyệt.
3. Gửi một yêu cầu HTTP POST chứa thông tin URL, Cookies, User-Agent và Tên tệp tin đến cổng **localhost:15151** - nơi máy chủ cục bộ chạy ngầm của ứng dụng Flow đang lắng nghe.
4. Ứng dụng Flow nhận dữ liệu và tự động hiện hộp thoại xác nhận tải xuống, mang lại trải nghiệm tiện lợi như IDM.

CHƯƠNG III: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

- **Quản lý tác vụ tải xuống:** Cho phép thêm mới liên kết, tạm dừng (pause), tiếp tục (resume), hủy bỏ (cancel) và xóa hoàn toàn tác vụ khỏi danh sách.
- **Tải xuống đa luồng:** Tự động chia tệp thành nhiều phân đoạn tùy chọn (1 đến 16 luồng) để tải xuống song song, tự động ghép nối (merge) các phần khi hoàn tất.
- **Hỗ trợ giao thức nâng cao:** Tải trực tiếp luồng video phân đoạn HLS (m3u8), tự động giải mã các luồng đa phương tiện.
- **Tích hợp trình duyệt:** Tự động bắt link thông qua cổng Local Server, hỗ trợ lưu trữ và chuyển giao Cookies, User-Agent từ trình duyệt để vượt qua cơ chế xác thực của máy chủ tải xuống.
- **Quản lý hàng đợi (Smart Queues):** Lập lịch thời gian bắt đầu/kết thúc tải, giới hạn tốc độ băng thông tối đa cho từng hàng đợi riêng biệt để tránh nghẽn mạng gia đình.
- **Cá nhân hóa giao diện:** Hỗ trợ 6 chủ đề màu sắc khác nhau, tối ưu chế độ hiển thị nền tối (OLED Black) và nền sáng (Industrial Light).

3.1.2 Yêu cầu phi chức năng

- **Hiệu năng:** Tốc độ tải xuống đạt tối thiểu 90% băng thông đường truyền tối đa của thiết bị. Thời gian ghép nối tệp tin dung lượng lớn (như tệp zip 2GB) dưới 10 giây nhờ tối ưu hóa cơ chế I/O của thư viện Okio.
- **Bảo mật & Riêng tư:** Không chứa telemetry, không gửi bất kỳ thông tin cá nhân hay lịch sử tải xuống nào về máy chủ từ xa (100% dữ liệu lưu trữ offline tại máy người dùng).
- **Tương thích đa nền tảng:** Hoạt động ổn định trên Windows 10/11, macOS (Intel & Apple Silicon), Linux (Ubuntu/Debian) và Android 8.0 trở lên.
- **Dung lượng bộ nhớ:** RAM chiếm dụng của ứng dụng nền trên Desktop không vượt quá 60MB.

3.2. Thiết kế kiến trúc

Hệ thống được thiết kế theo kiến trúc 3 tầng phân lớp rõ ràng nhằm giảm thiểu sự phụ thuộc lẫn nhau giữa các thành phần:

(Sơ đồ kiến trúc 3 tầng được thể hiện chi tiết bằng hình vẽ trong tệp tài liệu Word)

3.3. Sơ đồ và kịch bản Use Case

Use Case: Tải xuống tệp tin mới (thông qua Extension bắt link)

Bảng 3.1: Đặc tả Use Case "Tải xuống tệp tin mới"

Thuộc tính	Mô tả chi tiết
Tên Use Case	Tải xuống tệp tin mới thông qua Browser Extension
Tác nhân	Người dùng, Trình duyệt Web (Chrome/Edge/Brave)
Mô tả	Người dùng click vào một link tải trên trình duyệt, ứng dụng Flow tự động chặn link và thực hiện tải xuống đa luồng.
Điều kiện bắt đầu	Ứng dụng Flow đang chạy (hoặc chạy ngầm trong System Tray) và Browser Extension đã được bật.
Kịch bản chính	1. Người dùng click vào liên kết tải xuống trên trình duyệt. 2. Extension chặn sự kiện và gửi yêu cầu POST đến Local Server ở cổng 15151. 3. Local Server chuyển thông tin tải xuống cho Downloader Engine. 4. Một hộp thoại "Add Download" xuất hiện hiển thị: Tên tệp, Kích thước, Vị trí lưu. 5. Người dùng click "INITIALIZE". 6. Hệ thống phân đoạn file, tạo các kết nối HTTP song song và bắt đầu tải xuống.
Kịch bản phụ (Ngoại lệ)	- Nếu máy chủ không hỗ trợ tải đa luồng (HTTP header Accept-Ranges không có hoặc bằng none), hệ thống sẽ tự động chuyển sang cơ chế tải đơn luồng an toàn. - Nếu mất mạng, hệ thống tự động đưa tệp vào trạng thái "Pending" và thử lại (retry) sau mỗi 10 giây.

3.4. Thiết kế cấu trúc dữ liệu lưu trữ

Cấu hình ứng dụng được tổ chức lưu trữ phân mảnh dạng JSON Key-Value thông qua DataStore:

- 1. AppSettings (`app\_settings.json`):** Lưu trữ cấu hình ngôn ngữ, thư mục tải mặc định, số luồng tối đa mặc định, cấu hình khởi động cùng hệ thống.
- 2. ProxySettings (`proxy\_settings.json`):** Lưu cấu hình máy chủ trung gian (HTTP, SOCKS5) để người dùng có thể vượt qua tường lửa.

3. PerHostSettings (`per\_host\_settings.json`): Thiết lập giới hạn luồng hoặc tài khoản đăng nhập (basic auth) cho từng domain cụ thể.

CHƯƠNG IV: XÂY DỰNG HỆ THỐNG VÀ KẾT QUẢ ĐẠT ĐƯỢC

4.1. Quy trình xây dựng mã nguồn

Tác giả đã triển khai viết mã nguồn sạch bằng ngôn ngữ Kotlin. Dưới đây là các đoạn mã nguồn cốt lõi quan trọng nhất của hệ thống:

4.1.1. Hàm phân đoạn và tải xuống đa luồng song song (`HttpDownloadJob.kt`)

Đoạn mã này thực hiện tính toán kích thước của các chunk dựa trên dung lượng file và số luồng yêu cầu, sau đó sử dụng Coroutines để tải song song các phần:

```
// Downloader core: Thực thi tải xuống đa luồng song song
suspend fun startDownload(downloadItem: IDownloadItem) {
    val totalLength = downloadItem.totalLength
    val numParts = downloadItem.requestedParts // Ví dụ: 8 luồng
    val partSize = totalLength / numParts

    val jobs = (0 until numParts).map { partIdx ->
        val startByte = partIdx * partSize
        val endByte = if (partIdx == numParts - 1) totalLength - 1 else
(partIdx + 1) * partSize - 1

        // Chạy bất đồng bộ từng luồng tải chunk
        coroutineScope.launch(Dispatchers.IO) {
            downloadChunk(downloadItem.url, startByte, endByte, partIdx)
        }
    }
    jobs.joinAll() // Chờ tất cả các luồng hoàn tất
    mergeParts(downloadItem) // Ghép nối các file chunk thành file gốc
}
```

4.1.2. Máy chủ cục bộ tiếp nhận yêu cầu từ Extension (`shared:local-server`)

Đoạn mã khởi tạo máy chủ HTTP nhẹ chạy ngầm để hứng request từ Browser Extension:

```
// Local Server lắng nghe yêu cầu bắt link từ Extension
class ExtensionListenerServer(private val downloadManager: DownloadManager) {
    private val server = Http4kServer(port = 15151) { request ->
        if (request.method == Method.POST && request.uri.path == "/add-
download") {
            val body =
Json.decodeFromString<AddDownloadRequest>(request.bodyString())

            // Đẩy nhiệm vụ tải mới vào hàng đợi
            downloadManager.enqueue(body.url, body.filename, body.cookies)
            Response(Status.OK).body("Success")
        } else {
            Response(Status.NOT_FOUND)
        }
    }
}
```

```

fun start() = server.start()
fun stop() = server.stop()
}

```

4.2. Thử nghiệm hệ thống & Giao diện

Ứng dụng đã được biên dịch thành công ra các định dạng đóng gói chuẩn:

- **Windows:** Bộ cài .msi thông qua công cụ NSIS và Wix Toolset.
- **macOS:** Tập ảnh đĩa .dmg hỗ trợ cả hai kiến trúc chip Universal.
- **Linux:** Gói .deb cho các hệ điều hành Ubuntu/Debian.
- **Android:** Tập tin cài đặt .apk đã được ký số debug.

Mô tả thiết kế giao diện Cyber-Industrial (Brutalist Tech):

Giao diện của Flow được phát triển dựa trên triết lý tối giản kỹ thuật và góc cạnh sắc nét:

- Không sử dụng các góc bo tròn (BorderRadius = 0).
- Sử dụng font chữ monospaced JetBrains Mono để thể hiện dữ liệu tốc độ tải một cách rõ ràng, dễ nhìn nhất.
- Tông màu chủ đạo là đen sâu (#0A0A0A) kết hợp với các nút bấm góc cắt 45 độ (chamfered edges) màu cam neon tương phản cực cao (#E64A00), đem lại cảm giác hiện đại, đậm chất công nghiệp kỹ thuật số.

4.3. Đánh giá kết quả thực nghiệm

Để đánh giá tốc độ, tác giả đã thử nghiệm tải tập tin cài đặt có dung lượng **1.2 GB** từ máy chủ tốc độ cao của FPT. So sánh trực tiếp giữa trình tải mặc định của Google Chrome (1 luồng) và Flow Download Manager (sử dụng cấu hình 8 luồng tải song song).

Bảng 4.1: Kết quả thực nghiệm đo lường tốc độ và thời gian tải

Lần thử	Công cụ thử nghiệm	Băng thông lý thuyết	Tốc độ tải trung bình	Thời gian hoàn tất	Trạng thái ghép file
Lần 1	Chrome Downloader	100 Mbps	8.2 MB/s	2 phút 30 giây	N/A (Tải trực tiếp)
	Flow (8 luồng)	100 Mbps	11.8 MB/s	1 phút 44 giây	Thành công (1.8 giây)
Lần 2	Chrome Downloader	100 Mbps	7.9 MB/s	2 phút 35 giây	N/A

Flow (8 luồng)	100 Mbps	11.9 MB/s	1 phút 42 giây	Thành công (1.5 giây)
---------------------------	----------	------------------	---------------------------	----------------------------------

Nhận xét: Nhờ khả năng phân tách luồng, Flow đã tận dụng tối đa băng thông đường truyền mạng thực tế (gần sát giới hạn lý thuyết 12.5 MB/s của gói mạng 100 Mbps). Thời gian tải của Flow giảm gần **35%** so với trình duyệt mặc định, đồng thời thời gian ghép nối file cực kỳ nhanh chóng, không gây trễ cho người dùng.

KẾT LUẬN

Kết luận

Qua quá trình nghiên cứu lý thuyết, thiết kế và phát triển thực nghiệm hệ thống, đề tài "**Xây dựng ứng dụng quản lý và tăng tốc tải xuống Flow Download Manager đa nền tảng**" đã hoàn thành đầy đủ các mục tiêu đề ra:

- 1. Về mặt công nghệ:** Nghiên cứu và áp dụng thành công các công nghệ lập trình hiện đại nhất là Kotlin Multiplatform và Compose Multiplatform để xây dựng một ứng dụng đa nền tảng thực sự, chia sẻ tối đa mã nguồn logic và đồng nhất giao diện người dùng.
- 2. Về mặt chức năng:** Ứng dụng hoạt động ổn định trên cả máy tính để bàn (Windows, macOS, Linux) và thiết bị di động (Android). Các tính năng cốt lõi như tải đa luồng, tự động bắt link tải qua Extension, quản lý hàng đợi thông minh và tải luồng phát trực tuyến HLS đều hoạt động với hiệu năng rất tốt.
- 3. Về mặt thiết kế:** Giao diện Cyber-Industrial với phong cách Brutalist thô mộc, góc cạnh sắc nét, độ tương phản cao mang lại tính nhận diện thương hiệu độc đáo và trải nghiệm trực quan ấn tượng cho người dùng.

Hướng phát triển trong tương lai

Để ứng dụng ngày một hoàn thiện và phục vụ tốt hơn nhu cầu của cộng đồng, hướng phát triển tiếp theo của đề tài sẽ tập trung vào:

- Tích hợp giao thức tải xuống ngang hàng Peer-to-Peer (BitTorrent) trực tiếp vào downloader core.
- Xây dựng cơ chế quét mã độc tự động cho các tệp tin tải xuống để bảo vệ thiết bị của người dùng.
- Cải tiến Browser Extension để có khả năng bắt và giải mã các luồng phát video trực tuyến phức tạp hơn (sử dụng DRM mã hóa).
- Đưa ứng dụng lên các cửa hàng chính thức như Google Play Store và Mac App Store.

TÀI LIỆU THAM KHẢO

1. **Hoàng Thu Trang (2017)**, *Nghiên cứu và xây dựng ứng dụng giám sát hành trình trên điện thoại di động*, Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội.
2. **JetBrains Official Docs**, *Kotlin Multiplatform for shared business logic documentation*, Địa chỉ trực tuyến: <https://kotlinlang.org/docs/multiplatform.html> (Truy cập tháng 03/2026).
3. **JetBrains Official Docs**, *Compose Multiplatform UI framework documentation*, Địa chỉ trực tuyến: <https://www.jetbrains.com/lp/compose-multiplatform/> (Truy cập tháng 03/2026).
4. **Ktor HTTP Client Development Guide**, *Asynchronous network connections in Kotlin*, Địa chỉ trực tuyến: <https://ktor.io/docs/client.html> (Truy cập tháng 03/2026).
5. **Android Jetpack Architecture**, *DataStore Guide for local storage persistence*, Địa chỉ trực tuyến: <https://developer.android.com/topic/libraries/architecture/datastore> (Truy cập tháng 03/2026).
6. **Decompose Navigation Library Docs**, *Component-based navigation state keeper*, Địa chỉ trực tuyến: <https://arkivanov.github.io/Decompose/> (Truy cập tháng 03/2026).
7. **Chrome Developers Documentations (2025)**, *Manifest V3 extensions for chrome downloads api*, Địa chỉ trực tuyến: <https://developer.chrome.com/docs/extensions> (Truy cập tháng 03/2026).
8. **W3C Network Protocols Group**, *HTTP/1.1 Range Requests specifications (RFC 7233)*, Địa chỉ trực tuyến: <https://tools.ietf.org/html/rfc7233> (Truy cập tháng 03/2026).
9. **FPT Telecom Speed Tests**, *Network bandwidth statistics and limits under single and multi-thread connections*, (Truy cập tháng 03/2026).
10. **GitHub Repository Codebase**, *luuconghoangnam/flowspeed.link source code structure and analysis*, Địa chỉ trực tuyến: <https://github.com/luuconghoangnam/flowspeed.link> (Truy cập tháng 03/2026).